TASK 8 – APPLY POLYMORPHISM

1. Introduction

Polymorphism is an important concept in object-oriented programming that allows the same method name or reference to represent different behaviours. Java supports two major types of polymorphism: compile-time polymorphism and runtime polymorphism.

In the Campus Smart-Parking System, polymorphism allows different vehicle categories to calculate their parking fees using a common method name while applying their respective parking rates.

The system demonstrates both method overloading and method overriding to achieve flexible and reusable code.

2. Compile-Time Polymorphism – Method Overloading

Compile-time polymorphism is achieved through method overloading. It allows multiple methods in the same class to have the same name but different parameter lists.

In the parking system, the `calculateFee()` method can be overloaded to accept different types of input.

Example:

```
calculateFee(int minutes)
calculateFee(LocalTime in, LocalTime out)
```

The first method accepts the total parking duration in minutes, whereas the second method accepts the vehicle's entry and exit times.

The compiler determines which method to execute based on the number, types and order of arguments supplied.

Example implementation:

```
public double calculateFee(int minutes) {
    int hours = (minutes + 59) / 60;
    return hours * ratePerHour();
}

public double calculateFee(LocalTime in, LocalTime out) {
    long minutes = Duration.between(in, out).toMinutes();
    return calculateFee((int) minutes);
}
```

Both methods perform parking fee calculations but accept different input formats.

Therefore, method overloading improves flexibility and supports compile-time polymorphism.

3. Runtime Polymorphism – Method Overriding

Runtime polymorphism occurs when a superclass reference refers to a subclass object and the overridden method of the actual object is executed at runtime.

In the Campus Smart-Parking System, an abstract class named `Vehicle` acts as the superclass. The subclasses `TwoWheeler`, `Car` and `ElectricCar` provide specialized implementations of the parking rate method.

The following method is declared in the superclass:

```
protected abstract double ratePerHour();
```

The subclasses implement this method according to their parking rates.

TwoWheeler:

```
@Override
protected double ratePerHour() {
    return 10;
}
```

Car:

```
@Override
protected double ratePerHour() {
    return 30;
}
```

ElectricCar:

ElectricCar inherits the parking rate from Car and obtains additional charging functionality through the Chargeable interface.

The common `calculateFee()` method calls `ratePerHour()`. At runtime, Java executes the implementation belonging to the actual vehicle object.

This demonstrates runtime polymorphism through method overriding.

4. Upcasting – A Single Array of Vehicle References

Upcasting means assigning a subclass object to a superclass reference.

Java allows an array of superclass references to store objects belonging to different subclasses.

Example:

```
Vehicle[] parked = {
    new TwoWheeler("TN01A1"),
    new Car("TN02B2"),
    new ElectricCar("TN03C3")
};
```

Here, `parked` is an array of Vehicle references containing three different vehicle objects.

- The first element refers to a TwoWheeler object.
- The second element refers to a Car object.
- The third element refers to an ElectricCar object.

This approach allows different vehicle categories to be managed through a common superclass reference without creating separate arrays for every vehicle type.

5. Dynamic Method Dispatch

Dynamic method dispatch is the mechanism through which Java determines which overridden method should execute at runtime.

When an overridden method is called through a superclass reference, Java identifies the actual object type and executes its corresponding implementation.

Example:

```
for (Vehicle v : parked) {
```

```
    double fee = v.calculateFee(130);
    System.out.println(fee);
}
```

Although the reference type is `Vehicle`, the actual object may be a TwoWheeler, Car or ElectricCar.

Java automatically invokes the appropriate inherited or overridden method.

Trace of Dynamic Method Dispatch

Consider the following statement:

```
Vehicle v = new ElectricCar("TN03C3");
double fee = v.calculateFee(130);
```

Property	Value
Declared reference type	Vehicle
Actual object type	ElectricCar
Inherited superclass	Car
Parking rate used	Rs. 30 per hour
Billable duration	3 hours
Executed calculation	3 × 30
Parking fee	Rs. 90

Execution sequence:

1. The reference variable `v` is declared as type Vehicle.
2. An ElectricCar object is created and assigned to the reference.
3. The `calculateFee(130)` method is called.
4. The inherited calculation method obtains the rate through `ratePerHour()`.
5. The runtime object is ElectricCar.
6. ElectricCar inherits the rate implementation from Car.
7. The parking fee is calculated as Rs. 90.

This demonstrates dynamic method dispatch.

6. Interface-Based Polymorphism – Chargeable

The `Chargeable` interface represents an additional capability available to electric vehicles.

```
interface Chargeable {
    double chargingFee(int kWh);
}
```

The ElectricCar class implements this interface and provides its own charging fee calculation.

```
class ElectricCar extends Car implements Chargeable {

    ElectricCar(String regNo) {
        super(regNo);
    }

    @Override
```

```
    public double chargingFee(int kWh) {
        return kWh * 12;
    }
}
```

The interface separates charging behaviour from the general vehicle structure.

The parking fee and charging fee are calculated independently and then added to obtain the total amount.

For an ElectricCar consuming 10 kWh:

Charging fee = 10 × Rs. 12 = Rs. 120.

Parking fee = Rs. 90.

Total fee = Rs. 90 + Rs. 120 = Rs. 210.

7. Why Polymorphism Avoids Long if–else Chains

Without polymorphism, the system would need to check the vehicle type using multiple if–else statements.

For example:

```
if (type.equals("TwoWheeler")) {
    fee = hours * 10;
}
else if (type.equals("Car")) {
    fee = hours * 30;
}
else if (type.equals("ElectricCar")) {
    fee = hours * 30;
}
```

As more vehicle categories are introduced, the conditional statements become longer and more difficult to maintain.

With polymorphism, each subclass provides its own implementation. The system calls the common method through a Vehicle reference, and Java automatically selects the appropriate behaviour.

This improves code organization, reduces conditional logic and supports extensibility.

8. Alternative Concept – Using instanceof Checks and Casting

Instead of dynamic method dispatch, the system could use `instanceof` checks and explicit casting to identify the actual vehicle type.

Example:

```
double calculateFee(Vehicle v, int minutes) {

    int hours = (minutes + 59) / 60;

    if (v instanceof TwoWheeler) {
        return hours * 10;
    }
    else if (v instanceof ElectricCar) {
        return hours * 30;
    }
    else if (v instanceof Car) {
        return hours * 30;
    }

    throw new IllegalArgumentException(
        "Unknown vehicle type");
}
```

This approach checks the actual object type before applying the corresponding parking rate.

However, it requires explicit type checking and becomes more difficult to maintain as new vehicle categories are added.

Comparison Between Dynamic Dispatch and instanceof Checks

Criteria	Dynamic Method Dispatch	instanceof and Casting
Behaviour selection	Automatically selected at runtime	Explicit type checking
Implementation	Overridden methods in subclasses	Conditional statements
Code organization	Behaviour belongs to the relevant subclass	Behaviour is centralized
Extensibility	New subclasses can provide their own implementation	Existing conditional logic must be updated
Maintenance	Easier to maintain	Becomes more complex as types increase
Polymorphism	Directly supports runtime polymorphism	Relies on explicit type identification

Time Complexity Analysis

Let k represent the number of vehicle types.

Dynamic Method Dispatch:

- Java selects the appropriate overridden method using runtime dispatch.
- The method selection has constant-time complexity in the conceptual O(1) model used for this comparison.
- The parking fee calculation involves a constant number of arithmetic operations.
- Therefore, the time complexity per fee calculation is O(1).

instanceof Checks and Casting:

- The system checks vehicle types using conditional statements.
- In the worst case, it may perform up to k checks.
- Therefore, the worst-case selection complexity is O(k).
- The fee calculation itself requires O(1) time.

Effect on Code Size and Maintenance

With dynamic method dispatch, a new vehicle type can be introduced by creating a subclass and implementing the required method. Existing vehicle classes generally remain unchanged.

With `instanceof` checks, the central fee calculation method must be modified whenever a new vehicle type is added. This increases code size and maintenance effort.

Therefore, dynamic method dispatch provides a more scalable and maintainable approach for the parking system.

9. Explanation of the Last Output Line

The given program executes:

```
System.out.println(
    new Car("TN02B2").equals(
        new ElectricCar("TN02B2")
    )
);
```

The output is:

```
true
```

This occurs because the `equals()` method in the Vehicle superclass compares vehicle objects using their registration numbers.

Both objects have the same registration number, `TN02B2`. Therefore, the comparison returns true even though the objects belong to different subclasses.

Is this behaviour correct?

Yes, provided registration numbers are globally unique across all vehicle categories. A vehicle's registration number identifies the vehicle independently of whether it is a car or an electric car.

However, if the system allows the same registration number to represent different vehicles, the equality condition would need to be revised.

10. Conclusion

Polymorphism provides an effective way to manage different vehicle categories in the Campus Smart-Parking System.

Method overloading demonstrates compile-time polymorphism, while method overriding and dynamic method dispatch demonstrate runtime polymorphism. Upcasting allows different vehicle objects to be stored in a single Vehicle reference array.

The Chargeable interface provides additional charging functionality for electric vehicles without modifying the common vehicle structure.

Compared with explicit instanceof checks and casting, dynamic method dispatch reduces conditional logic, improves maintainability and simplifies the addition of new vehicle categories.

Therefore, polymorphism is an appropriate design approach for developing a flexible, reusable and extensible Campus Smart-Parking System.